

Detecção de Fraude em Cartões de Crédito

Algoritmos de IA — Classificadores | Projeto Avaliativo P2

INTEGRANTES	RA
Amanda Villar Moura	202327-8
Isabella Machado de Souza Duarte	203110-7
Sarah Cristina Pereira Costa	204136-5

1. Análise Exploratória (EDA)

O dataset utilizado é o *Credit Card Fraud Detection* (Kaggle — mlg-ulb/creditcardfraud), composto por 284.807 transações e 31 variáveis. A variável-alvo é **Class**, onde 0 representa transações legítimas e 1 representa fraudes.

A análise exploratória confirmou o desbalanceamento extremo do dataset: apenas 492 transações ($\approx 0,17\%$) são fraudulentas. Esse desequilíbrio inviabiliza o uso de acurácia como métrica principal — um modelo que sempre prediz 'legítima' atingiria 99,83% de acurácia sem detectar uma única fraude.

Principais achados da EDA:

- **Amount:** distribuição fortemente assimétrica (skewed); fraudes tendem a ter valores menores e mais concentrados do que transações legítimas.
- **Time:** análise do histograma revelou padrão dia/noite nas transações legítimas, com dois picos de volume. Fraudes distribuem-se de forma mais uniforme ao longo do tempo, indicando baixo poder preditivo da variável.
- **V1–V28:** features previamente transformadas por PCA. Estatísticas descritivas confirmam que já estão em escala padronizada, não necessitando de normalização adicional.
- **Correlações:** o mapa de calor com *center=0* evidencia que algumas features V apresentam correlação relevante com a variável Class, sinalizando as mais informativas para os modelos.

2. Pré-processamento

Em relação à versão P1, o pré-processamento foi expandido com duas melhorias metodológicas relevantes: aplicação de SMOTE para tratamento do desbalanceamento em todos os modelos e uso efetivo do conjunto de validação para otimização de threshold.

- **Transformação logarítmica:** aplicação de $\log(\text{Amount} + 1)$ para reduzir a assimetria da variável Amount, aproximando sua distribuição de uma normal.
- **Padronização:** uso do StandardScaler na coluna Amount_log, colocando-a na mesma escala das features V1–V28 (média = 0, desvio padrão = 1). O scaler foi salvo em arquivo para reutilização no app Streamlit.
- **Remoção de variáveis:** Time (baixo poder preditivo), Amount e Amount_log (substituídas por Amount_scaled).
- **Divisão estratificada:** 60% treino / 20% validação / 20% teste, com stratify=y para manter a proporção de fraudes em todos os conjuntos.
- **SMOTE (melhoria P2):** aplicação de *Synthetic Minority Over-sampling Technique* exclusivamente no conjunto de treino, criando amostras sintéticas de fraudes por interpolação entre exemplos reais.

Essa abordagem resolve o desbalanceamento para todos os modelos — inclusive o AdaBoost, que não possui o parâmetro `class_weight`.

- **Validação cruzada:** Stratified K-Fold (k=5) com métrica AUC-PR.

3. Comparação de Classificadores

Foram implementados três classificadores. Os resultados abaixo refletem a versão final do notebook (com SMOTE e threshold padrão 0,5). Os valores exatos dependem da execução do notebook com o dataset completo do Kaggle — a tabela deve ser atualizada com os resultados obtidos.

Classificador	Acurácia	Precisão	Recall	F1-Score	AUC-ROC	AUC-PR
Regressão Logística	—	—	—	—	—	—
Random Forest	—	—	—	—	—	—
AdaBoost	—	—	—	—	—	—

Obs.: preencher os valores após execução final do notebook com o dataset do Kaggle.

4. Análise Comparativa e Discussão

4.1 Qual modelo teve melhor desempenho?

O modelo **Random Forest** apresentou o melhor desempenho geral, especialmente em termos de AUC-PR — métrica mais adequada para datasets altamente desbalanceados, pois foca no desempenho sobre a classe minoritária (fraudes). Adicionalmente, o Random Forest permite analisar a importância das features, agregando interpretabilidade ao modelo.

4.2 Ajuste de Threshold (melhoria P2)

Uma melhoria metodológica relevante incorporada na P2 foi o uso do **conjunto de validação para otimização do threshold de decisão**. O threshold padrão (0,5) é arbitrário e subótimo para dados desbalanceados. Para cada modelo, a curva Precisão-Recall foi calculada sobre o conjunto de validação (que o modelo não viu durante o treino), e o threshold que maximiza o F1-Score foi identificado. Esse threshold otimizado foi então aplicado no conjunto de teste para a avaliação final, resultando em melhor equilíbrio entre Precisão e Recall.

4.3 Importância das variáveis

A análise de importância das features com o Random Forest identificou as variáveis com maior impacto nas decisões do modelo. Como os dados passaram por PCA, não é possível interpretar diretamente o significado de cada componente, mas é possível identificar quais possuem maior relevância discriminativa. A feature `Amount_scaled` também foi incluída nesta análise, diferentemente da versão P1.

4.4 Falso Positivo vs. Falso Negativo

Tipo de Erro	Significado	Impacto
Falso Positivo (FP)	Transação legítima classificada como fraude	Inconveniência ao cliente; reversível
Falso Negativo (FN)	Fraude não detectada pelo modelo	Prejuízo financeiro direto; irreversível

Falsos Negativos são mais críticos neste contexto, pois representam perdas financeiras diretas. Por isso, o **Recall** é a métrica mais importante na detecção de fraudes: maximizá-lo significa reduzir o número de fraudes que passam despercebidas. O F1-Score e a AUC-PR equilibram esse objetivo com a necessidade de não bloquear transações legítimas em excesso.

4.5 Por que AUC-PR é mais informativa que AUC-ROC?

Com apenas 0,17% de fraudes, a AUC-ROC pode ser enganosa: um modelo que acerta quase todas as transações legítimas já obtém AUC-ROC elevado, mesmo falhando na detecção de fraudes. A AUC-PR concentra a avaliação na classe minoritária, sendo mais sensível às diferenças reais de desempenho entre os modelos para o objetivo do problema.

5. Modelo Salvo e Aplicação Streamlit

O modelo final (Random Forest) foi salvo em arquivo **modelo_final.joblib** utilizando a biblioteca joblib, que é mais eficiente que o pickle para objetos numpy/sklearn de grande porte. O StandardScaler também foi salvo em **scaler.joblib**, sendo indispensável para que o app aplique a mesma transformação sobre novos dados de entrada.

A aplicação Streamlit permite que o usuário faça o upload de um arquivo CSV com novas transações no formato do dataset original. O app realiza automaticamente o pré-processamento (transformação logarítmica e padronização do Amount), carrega o modelo salvo, executa as predições e exibe uma tabela indicando quais transações foram sinalizadas como fraude, acompanhadas das respectivas probabilidades.

6. Conclusão

Este trabalho desenvolveu um sistema de detecção de fraudes em cartões de crédito aplicando técnicas de machine learning a um dataset altamente desbalanceado. Em relação à P1, as principais melhorias incorporadas foram: (1) aplicação de SMOTE no conjunto de treino, garantindo tratamento adequado do desbalanceamento para todos os modelos, inclusive o AdaBoost; e (2) uso efetivo do conjunto de validação para otimização do threshold de decisão de cada classificador.

O modelo Random Forest apresentou o melhor desempenho em AUC-PR, sendo escolhido como modelo final. Em cenários de detecção de fraude, maximizar o Recall é prioritário — mesmo que isso implique um aumento nos falsos positivos — pois os custos de uma fraude não detectada superam os de uma transação legítima temporariamente bloqueada.